

Standard Template Library (STL)

Object Oriented Programming

CSE-4301

Presented by

Rafsanjany Kushol

Assistant Professor



Standard Template Library (STL)



Standard Template Library (STL)

✓ Containers

Manage collections of objects of a certain kind

Two types: Sequence and Associative

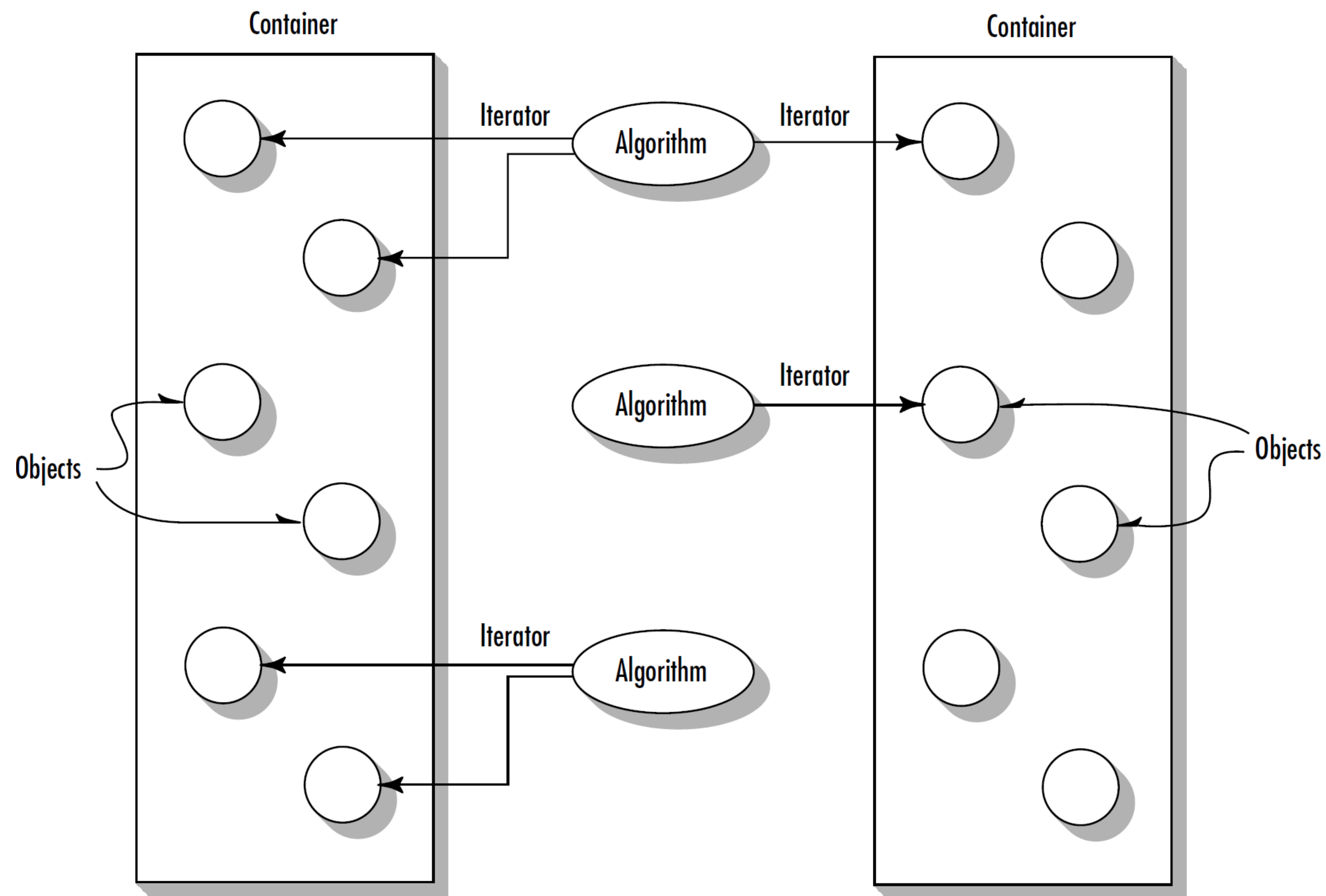
✓ Iterators

Used to step through the elements of collections of objects

✓ Algorithms

Used to process elements of collections

Standard Template Library (STL)



Algorithms use iterators to act on objects in containers

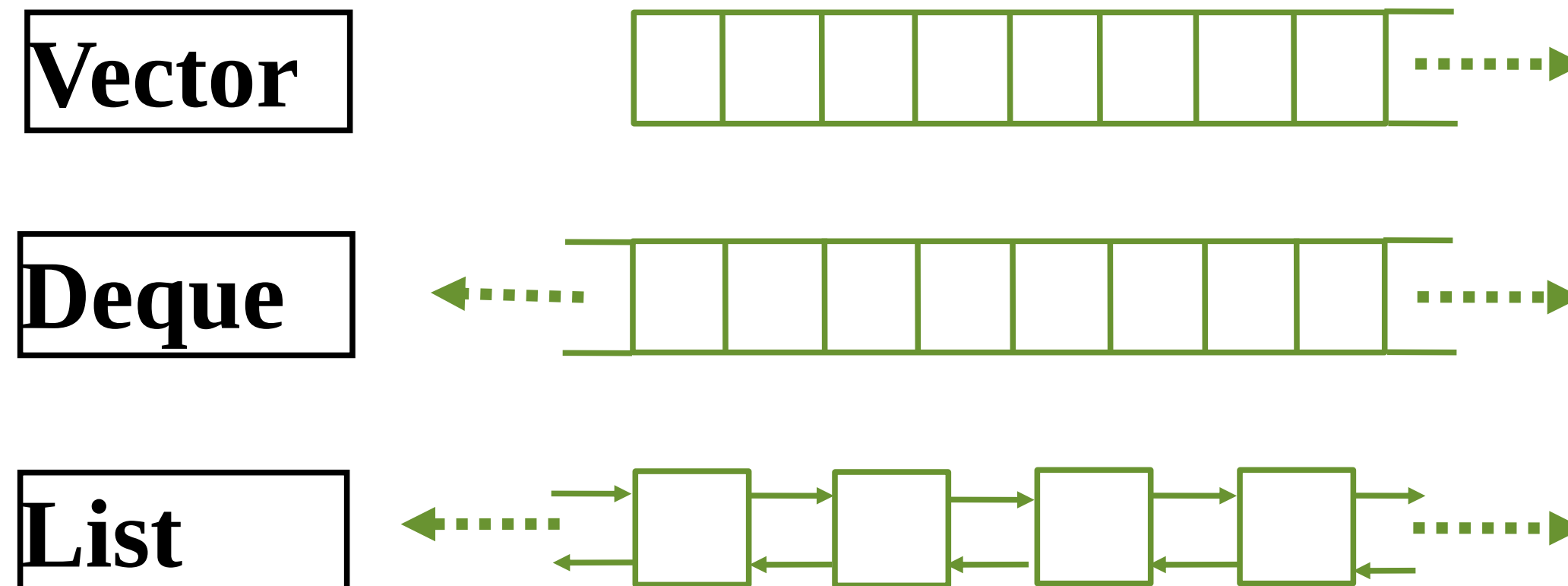
STL Sequence Containers

- Ordered Collections
- Every element has a certain position
- Position depends on order and place of insertion

Vector

Deque

List



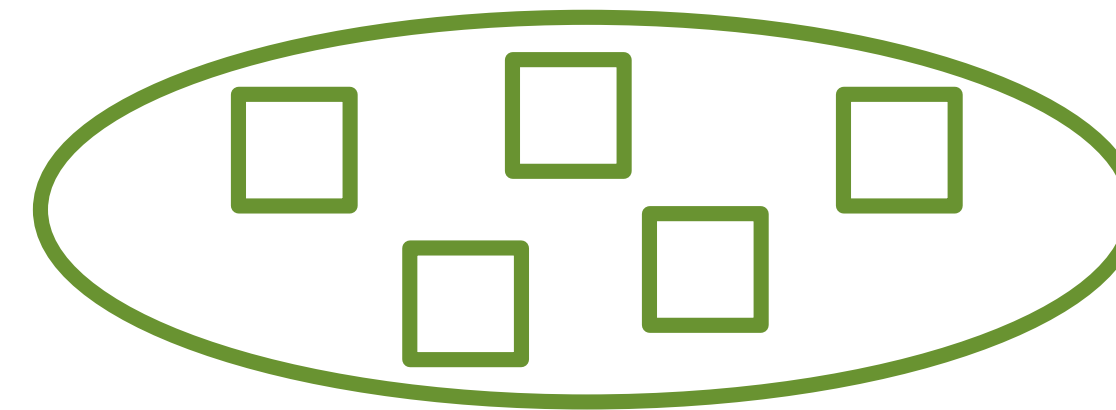
STL Associative Containers

- Not sequential; it uses keys to access data
- Actual position of an element depends on its value due to certain sorting criterion.
- Order of insertion doesn't matter.

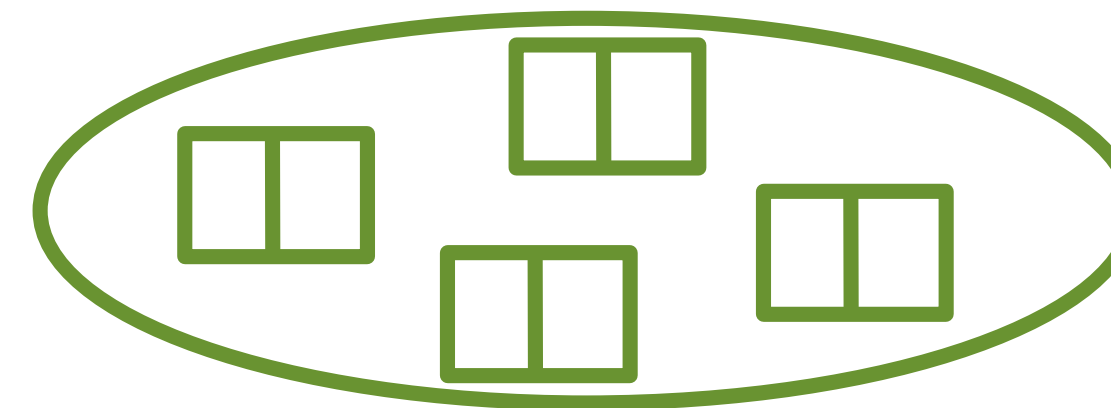
Set, Multiset

Map, Multimap

Set/Multiset



Map/Multimap



Member functions

<i>Name</i>	<i>Purpose</i>
<code>size()</code>	Returns the number of items in the container
<code>empty()</code>	Returns true if container is empty
<code>max_size()</code>	Returns size of the largest possible container
<code>begin()</code>	Returns an iterator to the start of the container, for iterating forwards through the container
<code>end()</code>	Returns an iterator to the past-the-end location in the container, used to end forward iteration
<code>rbegin()</code>	Returns a reverse iterator to the end of the container, for iterating backward through the container
<code>rend()</code>	Returns a reverse iterator to the beginning of the container; used to end backward iteration

STL Algorithms

<i>Algorithm</i>	<i>Purpose</i>
find	Returns first element equivalent to a specified value
count	Counts the number of elements that have a specified value
equal	Compares the contents of two containers and returns true if all corresponding elements are equal
search	Looks for a sequence of values in one container that corresponds with the same sequence in another container
copy	Copies a sequence of values from one container to another (or to a different location in the same container)
swap	Exchanges a value in one location with a value in another
iter_swap	Exchanges a sequence of values in one location with a sequence of values in another location
fill	Copies a value into a sequence of locations
sort	Sorts the values in a container according to a specified ordering
merge	Combines two sorted ranges of elements to make a larger sorted range
accumulate	Returns the sum of the elements in a given range
for_each	Executes a specified function for each element in the container

The find() Algorithm

```
// find.cpp
// finds the first object with a specified value
#include <iostream>
#include <algorithm>           //for find()
using namespace std;

int arr[] = { 11, 22, 33, 44, 55, 66, 77, 88 };

int main()
{
    int* ptr;
    ptr = find(arr, arr+8, 33);           //find first 33
    cout << "First object with value 33 found at offset "
          << (ptr-arr) << endl;
    return 0;
}
```

The output from this program is

First object with value 33 found at offset 2.

The count() Algorithm

```
// count.cpp
// counts the number of objects with a specified value
#include <iostream>
#include <algorithm>           //for count()
using namespace std;

int arr[] = { 33, 22, 33, 44, 33, 55, 66, 77 };

int main()
{
    int n = count(arr, arr+8, 33); //count number of 33's
    cout << "There are " << n << " 33's in arr." << endl;
    return 0;
}
```

The output is

There are 3 33's in arr.

The sort() Algorithm

```
// sort.cpp
// sorts an array of integers
#include <iostream>
#include <algorithm>
using namespace std;

                                //array of numbers
int arr[] = {45, 2, 22, -17, 0, -30, 25, 55};

int main()
{
    sort(arr, arr+8);           //sort the numbers

    for(int j=0; j<8; j++)      //display sorted array
        cout << arr[j] << ' ';
    cout << endl;
    return 0;
}
```

The output from the program is -30, -17, 0, 2, 22, 25, 45, 55

The search() Algorithm

```
// search.cpp
// searches one container for a sequence in another container
#include <iostream>
#include <algorithm>
using namespace std;

int source[] = { 11, 44, 33, 11, 22, 33, 11, 22, 44 };
int pattern[] = { 11, 22, 33 };

int main()
{
    int* ptr;
    ptr = search(source, source+9, pattern, pattern+3);

    if(ptr == source+9)                //if past-the-end
        cout << "No match found\n";
    else
        cout << "Match at " << (ptr - source) << endl;
    return 0;
}
```

The output is

Match at 3

The merge() Algorithm

```
// merge.cpp
// merges two containers into a third
#include <iostream>
#include <algorithm>          //for merge()
using namespace std;

int src1[] = { 2, 3, 4, 6, 8 };
int src2[] = { 1, 3, 5 };
int dest[8];

int main()
{
    //merge src1 and src2 into dest
    merge(src1, src1+5, src2, src2+3, dest);
    for(int j=0; j<8; j++)    //display dest
        cout << dest[j] << ' ';
    cout << endl;
    return 0;
}
```

The output, which displays the contents of the destination container, looks like this:

1 2 3 3 4 5 6 8

Function Objects

```
// sortemp.cpp
// sorts array of doubles in backwards order,
// uses greater<>() function object
#include <iostream>
#include <algorithm>           //for sort()
#include <functional>         //for greater<>
using namespace std;

                                // array of doubles
double fdata[] = { 19.2, 87.4, 33.6, 55.0, 11.5, 42.2 };

int main()
{
    // sort the doubles
    sort( fdata, fdata+5, greater<double>() );

    // display sorted doubles
    for(int j=0; j<6; j++)
        cout << fdata[j] << ' ';
    cout << endl;
    return 0;
}
```

The output:

87.4 55 42.2 33.6 19.2 11.5

Adding `_if` to Algorithms

```
// find_if.cpp
// searches array of strings for first name that matches "Don"
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;
//-----
bool isDon(string name)      //returns true if name=="Don"
{
    return name == "Don";
}
//-----
string names[] = { "George", "Estelle", "Don", "Mike", "Bob" };

int main()
{
    string* ptr;
    ptr = find_if( names, names+5, isDon );

    if(ptr==names+5)
        cout << "Don is not on the list.\n";
    else
        cout << "Don is element "
              << (ptr-names)
              << " on the list.\n";
    return 0;
}
```

The output from the program is

Don is element 2 on the list.

The for_each() Algorithm

```
// for_each.cpp
// uses for_each() to output inches array elements as centimeters
#include <iostream>
#include <algorithm>
using namespace std;

void in_to_cm(double);      //declaration

int main()
{
    //array of inches values
    double inches[] = { 3.5, 6.2, 1.0, 12.75, 4.33 };
    //output as centimeters
    for_each(inches, inches+5, in_to_cm);
    cout << endl;
    return 0;
}

void in_to_cm(double in)    //convert and display as centimeters
{
    cout << (in * 2.54) << ' ';
}
```

The output looks like this: 8.89 15.748 2.54 32.385 10.9982

The transform() Algorithm

```
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    //array of inches values
    double inches[] = { 3.5, 6.2, 1.0, 12.75, 4.33 };
    double centi[5];
    double in_to_cm(double);    //prototype
                                //transform into array centi[]
    transform(inches, inches+5, centi, in_to_cm);

    for(int j=0; j<5; j++)    //display array centi[]
        cout << centi[j] << ' ';
    cout << endl;
    return 0;
}

double in_to_cm(double in)    //convert inches to centimeters
{
    return (in * 2.54);        //return result
}
```

The output is the same as that from the FOR_EACH program.

Vectors

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v;

    v.push_back(10);
    v.push_back(11);
    v.push_back(12);
    v.push_back(13);

    v[0] = 20;
    v[3] = 23;

    for(int j=0; j<v.size(); j++)
        cout << v[j] << ' ';
    cout << endl;
    return 0;
}
```

The output is:

20 11 12 23

Vectors

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    //an array of doubles
    double arr[] = { 1.1, 2.2, 3.3, 4.4 };

    vector<double> v1(arr, arr+4); //initialize vector to array
    vector<double> v2(4);          //empty vector of size 4

    v1.swap(v2);                  //swap contents of v1 and v2

    while( !v2.empty() )
    {
        cout << v2.back() << ' ';
        v2.pop_back();
    }
    cout << endl;
    return 0;
}
```

The output is

4.4, 3.3, 2.2, 1.1

Vectors

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    int arr[] = { 100, 110, 120, 130 }; //an array of ints

    vector<int> v(arr, arr+4);          //initialize vector to array

    cout << "\nBefore insertion: ";
    for(int j=0; j<v.size(); j++)        //display all elements
        cout << v[j] << ' ';

    v.insert( v.begin()+2, 115);         //insert 115 at element 2

    cout << "\nAfter insertion: ";
    for(j=0; j<v.size(); j++)            //display all elements
        cout << v[j] << ' ';

    v.erase( v.begin()+2 );              //erase element 2

    cout << "\nAfter erasure: ";
    for(j=0; j<v.size(); j++)            //display all elements
        cout << v[j] << ' ';

    cout << endl;
    return 0;
}
```

The output:

Before insertion: 100 110 120 130

The output:

After insertion: 100 110 115 120 130

The output:

After erasure: 100 110 120 130

Lists

```
#include <iostream>
#include <list>
using namespace std;

int main()
{
    int j;
    list<int> list1, list2;

    int arr1[] = { 40, 30, 20, 10 };
    int arr2[] = { 15, 20, 25, 30, 35 };
```

```
    for(j=0; j<4; j++)
        list1.push_back( arr1[j] );
    for(j=0; j<5; j++)
        list2.push_back( arr2[j] );

    list1.reverse();
    list1.merge(list2);
    list1.unique();

    int size = list1.size();
    while( !list1.empty() )
    {
        cout << list1.front() << ' ';
        list1.pop_front();
    }
    cout << endl;
    return 0;
}
```

10, 15, 20, 25, 30, 35, 40

Deque

```
#include <iostream>
#include <deque>
using namespace std;

int main()
{
    deque<int> deq;

    deq.push_back(30);           //push items on back
    deq.push_back(40);
    deq.push_back(50);
    deq.push_front(20);         //push items on front
    deq.push_front(10);

    deq[2] = 33;                //change middle item

    for(int j=0; j<deq.size(); j++)
        cout << deq[j] << ' ';    //display items
    cout << endl;
    return 0;
}
```

The output of this program is

10 20 33 40 50

Associative Containers (Set)

```

int main()
{
    // array of string objects
    string names[] = {"Juanita", "Robert",
                     "Mary", "Amanda", "Marie"};
    // initialize set to array
    set<string, less<string> > nameSet(names, names+5);
    // iterator to set
    set<string, less<string> >::iterator iter;

    nameSet.insert("Yvette"); // insert more names
    nameSet.insert("Larry");
    nameSet.insert("Robert"); // no effect; already in set
    nameSet.insert("Barry");
    nameSet.erase("Mary");    // erase a name
    // display size of set

    cout << "\nSize=" << nameSet.size() << endl;
    iter = nameSet.begin();    // display members of set
    while( iter != nameSet.end() )
        cout << *iter++ << '\n';
}

```

Output:
 Size=7
 Amanda
 Barry
 Juanita
 Larry
 Marie
 Robert
 Yvette

Associative Containers (Set)

```
string searchName;           // get name from user
cout << "\nEnter name to search for: ";
cin >> searchName;

                               // find matching name in set
iter = nameSet.find(searchName);
if( iter == nameSet.end() )
    cout << "The name " << searchName << " is NOT in the set.";
else
    cout << "The name " << *iter << " IS in the set.";
cout << endl;
```


Associative Containers (Set)

```
int main()
{
    set<string, less<string> > organic;
    set<string, less<string> >::iterator iter;

    organic.insert("Culine"); // insert organic compounds
    organic.insert("Xanthine");
    organic.insert("Curarine");
    organic.insert("Melamine");
    organic.insert("Cyanimide");
    organic.insert("Phenol");
    organic.insert("Aphrodine");
    organic.insert("Imidazole");
    organic.insert("Cinchonine");
    organic.insert("Palmitamide");
    organic.insert("Cyanimide");

    iter = organic.begin(); // display set
    while( iter != organic.end() )
        cout << *iter++ << '\n';
}
```

Output:

Aphrodine
Cinchonine
Curarine
Culine
Cyanimide
Imidazole
Melamine
Palmitamide
Phenol
Xanthine

Associative Containers (Set)

```
string lower, upper;           // display entries in range
cout << "\nEnter range (example C Czz): ";
cin >> lower >> upper;
iter = organic.lower_bound(lower);
while( iter != organic.upper_bound(upper) )
    cout << *iter++ << '\n';
```

Enter range (example C Czz): Aaa Curb

Aphrodine
Cinchonine
Curarine

Organic:

Aphrodine
Cinchonine
Curarine
Curine
Cyanimide
Imidazole
Melamine
Palmitamide
Phenol
Xanthine

Associative Containers (Map)

```
int main()
{
    string name;
    int pop;
    string states[] = { "Wyoming", "Colorado", "Nevada",
                        "Montana", "Arizona", "Idaho" };
    int pops[] = { 470, 2890, 800, 787, 2718, 944 };

    map<string, int, less<string> > mapStates; //map
    map<string, int, less<string> >::iterator iter; //iterator

    for(int j=0; j<6; j++)
    {
        name = states[j]; //get data from arrays
        pop = pops[j];
        mapStates[name] = pop; //put it in map
    }
    cout << "Enter state: "; //get state from user
    cin >> name;
    pop = mapStates[name]; //find population
    cout << "Population: " << pop << ",000\n";
}
```


Associative Containers (Map)

```
for (iter = mapStates.begin(); iter != mapStates.end(); iter++)  
    cout << (*iter).first << ' ' << (*iter).second << ",000\n";
```

Output:

Arizona 2718,000
Colorado 2890,000
Idaho 944,000
Montana 787,000
Nevada 800,000
Wyoming 470,000

Storing User-Defined Objects

```

person pers1 ("Deauville", "William", 8435150);
person pers2 ("McDonald", "Stacey", 3327563);
person pers3 ("Bartoski", "Peter", 6946473);
person pers4 ("KuangThu", "Bruce", 4157300);
person pers5 ("Wellington", "John", 9207404);
person pers6 ("McDonald", "Amanda", 8435150);
person pers7 ("Fredericks", "Roger", 7049982);
person pers8 ("McDonald", "Stacey", 7764987);

multiset< person, less<person> > persSet;
multiset<person, less<person> >::iterator iter;

cout << "\nNumber of entries = " << persSet.size();

iter = persSet.begin();    // display contents of multiset
while( iter != persSet.end() )
    (*iter++).display();

```


Storing User-Defined Objects

```

person pers1 ("Deauville", "William", 8435150);
person pers2 ("McDonald", "Stacey", 3327563);
person pers3 ("Bartoski", "Peter", 6946473);
person pers4 ("KuangThu", "Bruce", 4157300);
person pers5 ("Wellington", "John", 9207404);
person pers6 ("McDonald", "Amanda", 8435150);
person pers7 ("Fredericks", "Roger", 7049982);
person pers8 ("McDonald", "Stacey", 7764987);

```

Number of entries = 8

multiset < person, less <person>	Bartoski,	Peter	phone: 6946473
multiset <person, less <person>	Deauville,	William	phone: 8435150
	Fredericks,	Roger	phone: 7049982
	KuangThu,	Bruce	phone: 4157300
	McDonald,	Amanda	phone: 8435150
	McDonald,	Stacey	phone: 3327563
	McDonald,	Stacey	phone: 7764987
	Wellington,	John	phone: 9207404

Storing User-Defined Objects

Enter last name of person to search for: McDonald

Enter first name: Stacey

Number of persons with this name = 2

McDonald,	Stacey	phone: 3327563
-----------	--------	----------------

McDonald,	Stacey	phone: 7764987
-----------	--------	----------------

```
                                // get count of such persons
int cntPersons = persSet.count(searchPerson);
cout << "Number of persons with this name = " << cntPersons;

iter = persSet.lower_bound(searchPerson);
while( iter != persSet.upper_bound(searchPerson) )
    (*iter++).display();
cout << endl;
```

Storing User-Defined Objects

Similar example given using List

- listpers.cpp
- uses a list to hold person objects

Predefined Function Objects

TABLE 15.10 Predefined Function Objects

<i>Function Object</i>	<i>Return Value</i>
<code>T = plus(T, T)</code>	<code>x+y</code>
<code>T = minus(T, T)</code>	<code>x - y</code>
<code>T = times(T, T)</code>	<code>x*y</code>
<code>T = divide(T, T)</code>	<code>x/y</code>
<code>T = modulus(T, T)</code>	<code>x%y</code>
<code>T = negate(T)</code>	<code>-x</code>
<code>bool = equal_to(T, T)</code>	<code>x == y</code>
<code>bool = not_equal_to(T, T)</code>	<code>x != y</code>
<code>bool = greater(T, T)</code>	<code>x > y</code>
<code>bool = less(T, T)</code>	<code>x < y</code>
<code>bool = greater_equal(T, T)</code>	<code>x >= y</code>

Predefined Function Objects

```
class airtime
{
private:
    int hours;           //0 to 23
    int minutes;         //0 to 59
public:
    //default constructor
    airtime() : hours(0), minutes(0)
    { }

    //2-arg constructor
    airtime(int h, int m) : hours(h), minutes(m)
    { }

    void display() const //output to screen
    { cout << hours << ':' << minutes; }

    void get()           //input from user
    {
        char dummy;
        cout << "\nEnter airtime (format 12:59): ";
        cin >> hours >> dummy >> minutes;
    }
}
```


Predefined Function Objects

```

int main()
{
    char answer;
    airtime temp, sum;
    list<airtime> airlist;

    do {
        temp.get();
        airlist.push_back(temp);
        cout << "Enter another (y/n)? ";
        cin >> answer;
    } while (answer != 'n');

    //sum all the airtimes
    sum = accumulate( airlist.begin(), airlist.end(),
                      airtime(0, 0), plus<airtime>() );

    cout << "\nsum = ";
    sum.display();           //display sum
    cout << endl;
}

//overloaded + operator
airtime operator + (const airtime right) const
{
    //add members
    int temp_h = hours + right.hours;
    int temp_m = minutes + right.minutes;
    if(temp_m >= 60) //check for carry
    { temp_h++; temp_m -= 60; }
    return airtime(temp_h, temp_m); //return sum
}

```

Predefined Function Objects

Enter airtime (format 12:59) : 3:45

Enter another (y/n)? y

Enter airtime (format 12:59) : 5:10

Enter another (y/n)? y

Enter airtime (format 12:59) : 2:25

Enter another (y/n)? y

Enter airtime (format 12:59) : 0:55

Enter another (y/n)? n

sum = 12:15

Lab Final Project Presentation

**Week of March 10: Half of the groups (Team 1, 4, 5, 9) from 1A
Half of the groups (Team 6, 17, 20, 21, 22) from 1B**

**Week of March 17: Half of the groups (Team 15, 28, 29, 33) from 1A
Half of the groups (Team 24, 25, 27, 30, 34) from 1B**

Plus Lab viva on that day

No Lab Final exam

Remaining class plan

Thur/Fri, Feb 27/28: Lecture

Tuesday, Mar 4: Lecture

Thur/Fri, Mar 6/7: Lecture

Tuesday, March 11: Quiz 4

Thur/Fri, Mar 13/14: Lecture

Tuesday, March 17: Review if needed, exam paper distribution

Thur/Fri, Mar 20/21: : Review if needed, exam paper distribution



Thank You

Any questions?

kushol@iut-dhaka.edu

